Analys och simulering av virvelavlösning i realtid

INLÄMNING 1 : INLEDNING

Sixten Bohman, Ivar Messing

23 October 2026

1 Inledning

Aerodynamik är studien av luft i rörelse som interagerar med solida kroppar. Aerodynamik tillhör det mer generella området fluiddynamik som omfattar både vätskor och gaser, men i sig handlar det oftast om lyftkraft och luftmotstånd. Användningsområdena sträcker sig från design av broar och höga byggnader och energilösningar såsom vind och vattenkraft, till aeronautik och utformandet av farkoster. Ett historiskt exempel på applicerad aerodynamik är de tidiga försöken till flygning med hjälp av motordrivet luftfarkost (Sadrehaghighi, I (2026) *Aerodynamic Basics*. ResearchGate).

Virvelavlösning är ett fenomen inom fluidmekanik som uppstår när en fluid, såsom luft eller vatten, strömmar förbi ett hinder, till exempel en cylinder, vid vissa specifika flödesförhållanden. Om flödet är tillräckligt långsamt följer fluiden objektets form och strömmar jämnt förbi, men när hastigheten ökar förbi ett kritiskt värde blir flödet instabilt. Denna instabilitet innebär att fluiden inte längre kan följa objektets yta utan att flödet släpper från det. Då börjar flödet oscillera när det passerar objektet, och dessa oscillationer ger upphov till virvlar som omväxlande lösgör sig från objektets båda sidor och fortsätter bort med strömmen. Mönstret som bildas bakom objektet kallas Kármánvirvelgata, uppkallat efter fysikern Theodore von Kármán.

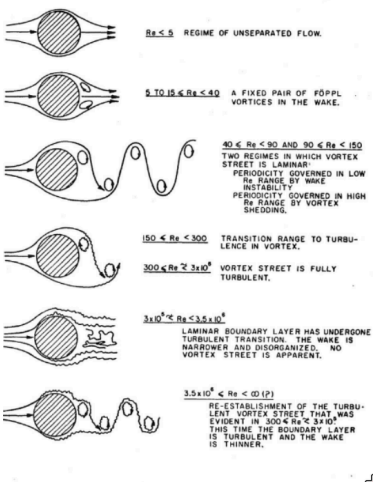


Fig. 1: Virvelavlösningens strömningsregimer

Vilka förhållanden som ger virvelavlösning beskrivs vanligtvis med det dimensionslösa Reynolds tal (Re), som sätter flödets hastighet, fluidens densitet och viskositet samt hindrets storlek i relation till varandra. Som visat i Figur 1 finns det olika stabilitetsområden i virvelavlösning för vissa Re (Lienhard, J. H. (1966), *Synopsis of Lift, Drag, and Vortex Frequency Data for Rigid Circular Cylinders*, WSU, Coll. of Engr. Bull. No. 300. Fig. 5). Virvelavlösningens frekvens beskrivs i sin tur av Strouhals tal, som är ungefär konstant över ett stort intervall av Reynolds tal.

Fenomenet förekommer också naturligt, till exempel när vindar passerar öar och bildar virvelmönster i molnen som kan observeras från satellit, och det är också orsaken till att luftledningar kan “sjunga” i vinden. Virvelavlösning har dessutom stor praktisk betydelse inom

ingenjörskonst, eftersom de växlande krafterna på ett hinder kan ge upphov till vibrationer i exempelvis skorstenar, broar och höga byggnader. Ytterligare används virvelavlösning bland annat i flödesvinkelsensorer och flödesmätare (Farsad, s., Parpanchi, S. M. & Rezaei, M. (2025) *Experimental study of vortex shedding phenomenon induced by various bluff body geometries for use in vortex flowmeters.* ScienceDirect.) Det är även vanligt att virvelavlösning uppstår i fluidsimuleringar, vilket gör det till ett lämpligt fenomen att använda för att undersöka hur väl simuleringar återger verkliga flöden (Weißmann, S. & Pinkall, U. (2010), *Filament-based smoke with vortex shedding and variational reconnection*, ACM, TU Berlin).

En simulering syftar till att återskapa en viss miljö så troget som möjligt. Genom att efterlikna ett verkligt eller tänkt system med hjälp av matematiska modeller kan man studera dess beteende utan att behöva utföra ett fysiskt experiment. Det är en avgörande metod inom många vetenskapsområden och inom datavetenskap, eftersom den gör det möjligt att undersöka situationer som annars vore för dyra, farliga eller omöjliga att testa i verkligheten. Användningsområdena sträcker sig från att simulera en realistisk värld i realtid för spelutveckling till att förutsäga en rymdrakets bana genom atmosfären. Simuleringar används framför allt för system som är så komplexa att en exakt återskapning är omöjlig, till exempel flöden, väder och rörelser i stora system av partiklar. Därför bygger metoden i grunden på välgrundade matematiska approximationer, där målet är att resultatet ska vara tillräckligt träffsäkert för sitt syfte snarare än perfekt.

Röksimuleringen körs i realtid eftersom de flesta beräkningarna är parallelliserade på grafikkortet. Det innebär att många beräkningar utförs samtidigt, vilket passar röksimuleringen väl eftersom varje del av röken kan beräknas i stort sett oberoende av de andra. Vi bygger en abstraktion över Vulkan, ett lågnivå-API för grafikkortsprogrammering, med hjälp av ett egetgjort ECS-baserat system i C++ (Bohman, S. (2026) *Moss Suite*, senaste commit [Computer Software]). ECS (Entity Component System) är ett sätt att strukturera program där data och logik hålls isär, vilket gör koden lättare att organisera och köra effektivt och snabbt. Syftet med abstraktionen är att tydliggöra parallelliseringens uppbyggnad och ändamål, samt att göra det enklare att ändra och uppgradera simuleringen i efterhand. Arbetet är tidskrävande, bland annat eftersom Vulkan är ett omständligt API där all hårdvara och alla optimeringsparametrar måste specificeras explicit. Det ger full kontroll över grafikkortet och hög prestanda, men kräver mycket kod och noggrannhet även för enkla operationer.

Navier-Stokes-ekvationerna är en samling differentialekvationer som beskriver rörelsen hos fluider. Vi är ur syfte av röksimulering intresserade av de okomprimerbara Navier-Stokes-ekvationerna som vi ser nedan (Bridson, J. & Müller-Fisher, M. (2007) *Fluid simulation: SIGGRAPH 2007 course notes.* ACM).

$$\begin{aligned} \frac{\partial \vec{u}}{\partial d} + \vec{u} \cdot \nabla \vec{u} + \frac{1}{\rho} \nabla p &= \vec{g} + \nu \nabla^2 \vec{u} \\ \nabla \cdot \vec{u} &= 0 \end{aligned} \tag{1}$$

Den första av dessa ekvationer, rörelsemängdsekvationen, beskriver hur fluiden accelereras av yttre krafter. Den mest elementära yttre kraften är gravitationen. Övriga krafter härrör samtliga från den omgivande vätskan. För att underlätta resonemanget approximeras vätskan här som en mängd små, ändliga partiklar. I själva simuleringen används visserligen ett semi-Euler-Lagrangiskt tillvägagångssätt, men av pedagogiska skäl är denna förenkling tillräcklig. Den första

av de inre krafterna uppstår genom att områden med högt tryck driver fluiden mot områden med lågt tryck. Detta beskrivs av termen $-\nabla p$, det negativa tryckgradienten, som i den fullständiga ekvationen divideras med densiteten ρ. Den andra inre kraften har sitt ursprung i vätskans viskositet. Intuitivt tenderar varje vätskepartikel att anta en hastighet som överensstämmer med omgivningens. Partikeln strävar således efter att minimera skillnaden mellan sin egen hastighet och hastigheten hos de omgivande partiklarna. Denna effekt beskrivs av Laplaceoperatorn ∇^2.

2 Metod

Vi använde en Vulkan-abstraktion i C++ för att skapa fluidsimuleringen. Abstraktionen använder ett ECS-baserat kärnsystem för att ge en ergonomisk, modulbaserad implementation som är användbar för många olika typer av rå shaderanvändning. Den abstraherar också bort många typer av interaktioner mellan shaders, såsom ping-pong-buffring och dubbelbuffrade bilder. Dess användning sträcker sig utanför just shadersimulering, men den är skriven för detta arbete och fungerar därför bäst för det.

Fluidsimuleringen använde fyra olika "simuleringsbilder": hastighet, tryck, divergens och slutligen färg, på engelska "dye". Färgen är den passiva skalär som till sist visas av simuleringen, och den motsvarar ungefär en punkts (rutas) densitet. Hastighet, tryck och färg är dubbelbuffrade, eftersom var och en uppdateras utifrån sitt eget tidigare värde. Divergensbilden är enkelbuffrad, eftersom den bara fungerar som mellanlagring: den skrivs i ett steg och läses först i efterföljande steg. Alla bilder använder en fast upplösning som är oberoende av fönsterstorleken.

Pipelinen består av flera beräkningspass som körs i en bestämd ordning för varje bildruta. Först beräknas hastighetsfältets divergens, som sedan används för att lösa tryckfältet med 20 Jacobi-iterationer. Tryckgradienten subtraheras därefter från hastighetsfältet för att minska divergensen och upprätthålla inkompressibilitet. Slutligen advecteras färgfältet med det korrigerade hastighetsfältet. Varje pass implementeras som en beräkningsshader (compute shader), där dubbelbuffring används för att undvika att data skrivs över innan den har lästs.

Tryckberäkningen löses med Jacobi-iterationer. Det är nödvändigt, eftersom hastighetsförändringen också leder till något som inte är fysikaliskt möjligt: fluiden trängs ihop på vissa ställen och glesnar på andra, trots att den i grunden är inkompressibel. Trycket används för att rätta till dessa avvikelser. Trycket i en cell beror på trycket hos grannarna, vilket bildar ett stort linjärt system som är opraktiskt att lösa direkt. Därför görs beräkningen i omgångar, så kallade Jacobi-iterationer, som successivt närmar sig ett svar. Vi utgår först från föregående bildrutas tryck, som fungerar som en bra startgissning, och efter varje iteration blir felet lite mindre. Vi använde 20 omgångar per bildruta, en avvägning mellan prestanda och noggrannhet (fler iterationer ger ett mer divergensfritt flöde).

Det som avgör om och i vilken form virvelavlösning uppstår är det så kallade Reynolds-talet, och det beror av objektets diameter, luftens flödeshastighet samt vätskans densitet och viskositet. Bland dessa egenskaper kan vi variera alla utom densiteten, då densiteten är inbakad i den simuleringens viskositet. För att undersöka om simuleringen kvalitativt återger virvelavlösning placerade vi ett cirkulärt objekt i flödet. Vi varierade sedan förutsättningarna i simulationen för att se ifall fenomenet kunde replikeras. För att sedan ta reda på ifall virvelavlösningen också

betedde sig på samma sätt som i verkligheten, där det bara uppstår i vissa intervall av Reynolds tal, justerade vi dessa variabler yttreligare.

3 Referenser

Farsad, S., Parpanchi, S. M. & Rezaei, M. (2025) *Experimental study of vortex shedding phenomenon induced by various bluff body geometries for use in vortex flowmeters.* ScienceDirect. https://www.sciencedirect.com/science/article/abs/pii/S099775462500007X

Bohman, S. (2026) *Moss Suite*, Senaste commit [Computer Software] https://github.com/snbohman/mosssuite.git

Bridson, J. & Müller-Fisher, M. (2007) *Fluid simulation: SIGGRAPH 2007 course notes.* ACM. https://dl.acm.org/doi/abs/10.1145/1281500.1281681

Weißmann, S. & Pinkall, U. (2010), *Filament-based smoke with vortex shedding and variational reconnection*, ACM, TU Berlin https://dl.acm.org/doi/10.1145/1778765.1778852

Sadrehaghighi, I (2026) *Aerodynamic Basics.* ResearchGate. https://www.researchgate.net/publication/339499388_Aerodynamic_Basics